

# Funções I: parâmetros, chamadas e return

## Questão 1 · Definir uma vez, chamar várias vezes

Regista as saídas e identifica os parâmetros e os argumentos de uma das chamadas. Quantas vezes executa o corpo da função?

O exemplo apresenta Início, Olá Ana, Olá Rui e Fim. Ao acrescentar uma chamada antes do print final, surge mais uma saudação nesse ponto. O corpo passa de duas para três execuções.

```
def saudar(nome, curso):  
    print('Olá', nome, '- curso:', curso)  
  
print('Início')  
saudar('Ana', 'Informática')  
saudar('Rui', 'Informática')  
saudar('Eva', 'Informática')  
print('Fim')
```

nome e curso são parâmetros. Na primeira chamada, Ana e Informática são argumentos. Os nomes pessoais usados pelos alunos podem variar.

## Questão 2 · Valores predefinidos e argumentos nomeados

Qual chamada existente mudou de resultado e porquê? Explica a diferença entre omitir taxa e fornecer taxa explicitamente.

Resultados iniciais: 90.0, 80.0 e 95.0. Com a taxa predefinida 0.15: 85.0, 80.0 e 95.0. Só a chamada que omite taxa usa o novo valor predefinido. Com taxa=0, o resultado é 100.

```
def aplicar_desconto(preco, taxa=0.15):  
    return preco * (1 - taxa)  
  
print(aplicar_desconto(100))  
print(aplicar_desconto(100, 0.20))  
print(aplicar_desconto(taxa=0.05, preco=100))  
print(aplicar_desconto(100, taxa=0))
```

# Funções I: parâmetros, chamadas e return

## Questão 3 · Mostrar ou devolver

Regista os valores de x e y antes e depois da alteração. Explica qual impressão desapareceu e como o resultado devolvido pode ser reutilizado.

Original: a função `mostrar_dobro` imprime 8; depois surgem x: None e y: 8. O `print(y + 3)` acrescenta 11.

Ao substituir `print(n * 2)` por `return n * 2` em `mostrar_dobro`, x passa a valer 8. Desaparece a impressão isolada de 8 dentro dessa função; continuam x: 8, y: 8 e 11. A atribuição recebe o valor devolvido pela chamada.

## Questão 4 · Uma função com decisão

Regista os resultados e explica o caso de igualdade. Quando o primeiro `return` executa, o segundo também executa?

```
def maior(a, b):  
    if a >= b:  
        return a  
    return b  
  
def menor(a, b):  
    if a <= b:  
        return a  
    return b  
  
print(maior(8, 3), menor(8, 3))  
print(maior(2, 9), menor(2, 9))  
print(maior(5, 5), menor(5, 5))
```

Saídas: 8 3; 9 2; 5 5. Em caso de igualdade, o valor devolvido é 5 em ambas. O primeiro `return` termina a chamada, pelo que o segundo não executa nessa chamada.

# Funções I: parâmetros, chamadas e return

## Questão 5 · Desafio final - uma solução possível

```
def area(base, altura):  
    return base * altura  
  
def perimetro(base, altura):  
    return 2 * (base + altura)  
  
base = float(input('Base: '))  
altura = float(input('Altura: '))  
print('Área:', area(base, altura))  
print('Perímetro:', perimetro(base, altura))
```

5 e 3: área 15.0, perímetro 16.0. 4 e 4: área 16.0, perímetro 16.0. 0 e 3: área 0.0, perímetro 6.0. O último é um caso-limite numérico aceite pelo enunciado. Cada função calcula uma grandeza; a interação fica no código principal.